MATHÉMATIQUES ET INFORMATIQUE
Sciences
Université Paris Cité

Plan de Tests

Programmation du Plakoto

Auteurs :
YA Hassrol
GHANIME Joseph
BENNAMANE Mahdi
REN Ziyi

Encadrant :
BOUZY Bruno

Groupe : L2N1

Table des matières

1 Introduction **2**
1.1 Objectifs du test 2
1.2 Documents de référence 2

2 Concepts de base **2**

3 Périmètre du test **2**

4 Stratégie de test **2**
4.1 Interface textuelle 2
4.2 Intelligence Artificielle (IA Monte Carlo) 3

5 Planning prévisionnel **3**

6 Environnement de test **3**
6.1 Environnement personnel 3
6.2 Environnement scolaire 4

7 Outils utilisés **4**

8 Critères d'acceptation **4**

9 Vérification de la documentation **4**

10 Annexes **5**

11 Glossaire **5**

12 Références **6**

1 Introduction

1.1 Objectifs du test

Ce document vise à organiser et définir la stratégie des tests du projet Plakoto. Les objectifs principaux sont de vérifier que le jeu implémente correctement les règles officielles, que l'interface textuelle est fonctionnelle, et que l'intelligence artificielle (IA), basée sur l'algorithme Monte Carlo, répond aux exigences de performance définies dans le cahier des charges.

1.2 Documents de référence

- Cahier des charges v2.2
- Document de Tests v1.0
- Cahier de Recette v1.0

2 Concepts de base

Cette section rappelle les principaux concepts que le lecteur doit connaître avant d'aborder ce document :

- **Règles du jeu Plakoto**
- **Interface textuelle**
- **Intelligence Artificielle (Monte Carlo)**
- **Types de tests : unitaires, intégration, fonctionnels**
- **Micro/Macro coups**
- **Simulation/Simulateur**

3 Périmètre du test

- Interface textuelle : Inclus
- Intelligence artificielle Monte Carlo : Inclus
- Interface graphique : Exclu (non implémentée)

4 Stratégie de test

Cette partie définit précisément la démarche adoptée pour effectuer les tests du projet, détaillant les procédures pour chacune des deux composantes principales du jeu : l'interface textuelle et l'intelligence artificielle.

4.1 Interface textuelle

Les tests seront réalisés manuellement afin de couvrir une large variété de scénarios :

- Choix de différents seeds (19, 1, 50, 100, 1234) pour garantir la reproductibilité et la variété des scénarios testés.
- Exécution complète de plusieurs parties pour chaque seed afin de vérifier :
  - L'affichage correct et cohérent de l'interface.
  - La prise en compte exacte des entrées utilisateurs.
  - Le respect strict des règles officielles du jeu.

- L'absence d'erreurs ou de comportements inattendus.

4.2 Intelligence Artificielle (IA Monte Carlo)

Les tests seront réalisés automatiquement afin de déterminer le paramètre optimal « ns » (nombre de simulations par coup) pour l'algorithme Monte Carlo, selon la procédure suivante :

- Effectuer des séries de confrontations directes entre deux IA Monte Carlo ayant des valeurs « ns » consécutives prédéfinies :
  - ns=1 contre ns=3
  - ns=3 contre ns=10
  - ns=10 contre ns=30
  - ns=30 contre ns=100
  - ns=100 contre ns=300
  - ns=300 contre ns=1000
- Pour chaque confrontation :
  - Réaliser 100 ou 1000 parties afin d'obtenir des résultats statistiquement significatifs.
  - Enregistrer le taux de victoire de l'IA ayant la valeur « ns » la plus élevée.
- Pour chaque valeur de « ns », mesurer systématiquement :
  - Le taux de victoire face à l'IA possédant un « ns » inférieur.
  - Le temps moyen nécessaire par coup, tout en veillant strictement à ce qu'il soit inférieur à **0.5 secondes.**
- Exploiter les résultats obtenus afin de définir précisément trois niveaux de difficulté pour le jeu final :
  - Facile : ns fixé à 30 (valeur déjà évaluée comme performante sans exiger une ressource importante)
  - Moyen : valeur intermédiaire optimisée entre ns=30 et la valeur optimale identifiée
  - Difficile : la valeur optimale finale (« max_ns ») offrant les meilleures performances sans dépasser les contraintes temporelles.

5 Planning prévisionnel

- Tests de l'interface textuelle : du 7 mars au 28 mars 2025 (Terminé)
- Tests de l'IA Monte Carlo : du 11 avril au début mai 2025
- Optimisation finale des paramètres IA : à partir du 2 mai 2025 (En cours, dates précises à compléter ultérieurement)

6 Environnement de test

6.1 Environnement personnel

- CPU : Intel Core i5-13420H, 12 threads, 2.10 GHz
- RAM : 16 GB
- GPU : Intel UHD Graphics Family
- OS : Windows 11 (64-bit)
- Python : 3.11.5

6.2 Environnement scolaire

— CPU : Intel Core i5-10500, 6 cœurs, 3.10 GHz
— RAM : 7.5 GB
— GPU : Intel UHD Graphics 630
— OS : Ubuntu 22.04.5 LTS
— Python : 3.10.14

7 Outils utilisés

— Python 3.x
— SVN (gestion de versions)
— PyCharm, VSCode (IDE)

8 Critères d'acceptation

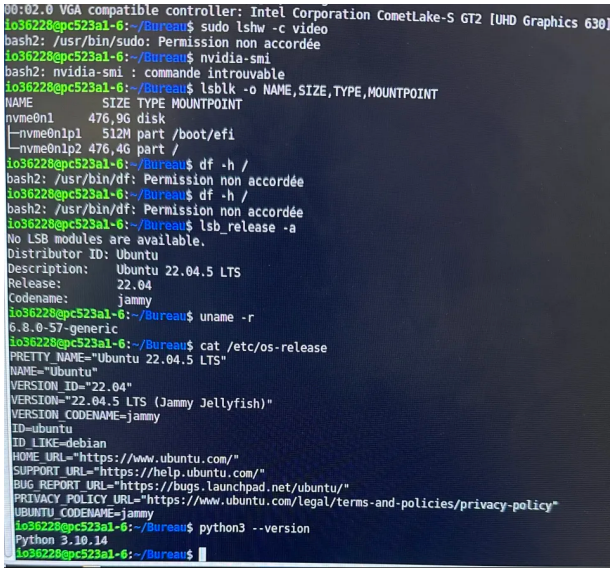
— Implémentation exacte des règles du jeu sans erreurs fonctionnelles.
— Interface textuelle intuitive et sans défaut majeur.
— Trois niveaux de difficulté clairement définis pour l'IA (Facile, Moyen, Difficile).
— Temps de réponse de l'IA inférieur à 0.5 secondes par coup.

9 Vérification de la documentation

Vérification complète des documents suivants :
— Manuel d'utilisation
— Manuel d'installation
— Document de Tests

10 Annexes

Procédure et captures d'écran du recueil des informations sur l'environnement de test (illustration du processus technique de collecte des environnements utilisés : configurations logicielles et matérielles détaillées)



- **Valeurs des « seeds » utilisées pour les tests :**
  - Hassrol : 19
  - Ziyi : 1
  - Mahdi : 50, 100
  - Ruby : 1234

11 Glossaire

- **Plakoto**
  Jeu traditionnel grec similaire au Backgammon, où l'objectif est de bloquer les pions adverses.
- **Intelligence Artificielle (IA)**
  Système informatique capable de prendre des décisions autonomes dans le jeu, ici basé sur l'algorithme Monte Carlo.
- **Monte Carlo**
  Méthode algorithmique utilisant des simulations aléatoires répétées pour estimer la meilleure décision à prendre dans un jeu ou un problème complexe.
- **Seed**
  Valeur initiale utilisée pour générer une séquence aléatoire reproductible lors des tests.
- **Test fonctionnel**
  Test visant à vérifier que les fonctionnalités du logiciel répondent aux spécifications définies.
- **Test unitaire**
  Test isolé d'une unité logicielle individuelle (fonction, méthode), afin de vérifier son bon fonctionnement de manière indépendante.

— **Bug**
  Défaut ou anomalie dans le logiciel qui entraîne un comportement inattendu ou incorrect.
— **SVN** (Subversion)
  Logiciel de gestion de versions permettant de gérer l'évolution des fichiers sources d'un projet informatique.
— **Micro-coup**
  Déplacement individuel d'un pion correspondant à l'un des deux dés lancés pendant un tour de jeu.
— **Macro-coup**
  Ensemble complet des déplacements réalisés par un joueur lors d'un tour, composé généralement de deux micro-coups.
— **Simulation/Simulateur**
  Processus algorithmique consistant à évaluer la qualité d'un coup potentiel en jouant virtuellement la partie jusqu'à son terme à partir d'un état donné du plateau. Le simulateur effectue cette évaluation en réalisant une série de parties automatisées dans lesquelles chaque joueur effectue des coups aléatoires (sans stratégie préétablie). Ces simulations répétées permettent à l'IA de déterminer statistiquement le taux de succès associé à chaque coup envisagé, afin de sélectionner la décision optimale.

12 Références

— **Cahier des charges v2.2** (Document définissant les exigences fonctionnelles et techniques du projet)
— **Documentation officielle Python 3.x** : https://docs.python.org/fr/3/
— **Méthode de Monte Carlo en Intelligence Artificielle** (Introduction théorique à l'approche Monte Carlo pour l'évaluation statistique des décisions dans les jeux)